Bounded Rescheduling for Flexible Job Shops via Spatiotemporal Reservation Closure

QI YU, Southeast University, PR China

WEI LI*, Southeast University, PR China

In flexible job shop scheduling that captures processing order with operation precedence and records corridor occupancies with time-window reservations, the rescheduling scope after a disturbance is still delineated by identifying the affected operations and their successors. When a corridor becomes impassable or slower during some time window, however, no operation becomes unexecutable: this approach returns an empty scope although the corridor occupancies inside the disturbed interval are already invalid, the original schedule is infeasible, and the resulting waits propagate to downstream vehicles through yielding relations. The gap lies not in detection but in the fact that the operation-precedence graph contains no yielding relations among corridor occupancies.

We define the rescheduling scope on corridor occupancies and propose the spatiotemporal reservation closure (STRC), whose result is called the reservation impact set. Starting from the reservations that overlap the disturbance window, STRC takes the transitive closure along four kinds of edges: yielding on the same corridor, and the next occupancy of the same vehicle, job, and machine. Occupancies inside the set may be rewritten; those outside it keep the original plan. Once the edges are fixed, membership in the set is uniquely determined and independent of the rerouting algorithm. Bounded rescheduling then repairs only inside the set: it reroutes the original vehicles and, if the schedule remains infeasible, enlarges the rewritable scope in a prescribed order.

On 50 instance–seed pairs, the scope on the operation-precedence graph is empty under corridor blockage on 50/50 pairs, while the original schedule is infeasible on all of them; with the rerouting procedure unchanged and only the scope object replaced, the feasibility rate rises from 0/50 to 50/50. The impact set is closed under outgoing edges on 50/50 pairs, and every occupancy outside it is unchanged on 50/50 pairs. Rewriting all unfinished occupancies instead leaves runtime and makespan almost unchanged but raises the rewrite fraction from 0.540 to 0.604 (R2 fewer/equal/more rewrites in 31/18/1 cells; Wilcoxon $p = 8.8 \times 10^{-7}$). Compared with warm-started global rescheduling, bounded repair is two to three orders of magnitude faster but has a worse makespan at every budget level. The contribution lies in the scope object; rerouting inside the set and enlarging the scope after failure follow existing practice.

CCS Concepts: • **Theory of computation** → **Scheduling algorithms**; • **Applied computing** → *Industry and manufacturing*; • **Computing methodologies** → *Planning and scheduling*.

Additional Key Words and Phrases: flexible job shop scheduling, bounded rescheduling, conflict-free AGV routing, spatiotemporal reservation table, spatiotemporal reservation closure, corridor blockage

ACM Reference Format:

Qi Yu and Wei Li. 2026. Bounded Rescheduling for Flexible Job Shops via Spatiotemporal Reservation Closure. 1, 1 (October 2026), 21 pages. https://doi.org/XXXXXXX.XXXXXXX

*corresponding author

Authors' Contact Information: Qi Yu, 230240012@seu.edu.cn, School of Cyber Science and Engineering, Key Laboratory of Computer Network and Information Integration, Southeast University, Nanjing, Jiangsu, PR China; Wei Li, xchlw@seu.edu.cn, School of Computer Science and Engineering, Key Laboratory of Computer Network and Information Integration, Southeast University, Nanjing, Jiangsu, PR China.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

Manuscript submitted to ACM

1 Introduction

Flexible job shop scheduling with transportation has long assumed that the time to move a job between two machines is a constant read from a matrix indexed by location pairs, which implicitly holds that vehicles never compete for the same corridor segment. Once a finite fleet shares a sparse corridor network and reserves contested segments with time windows, travel time becomes the outcome of two decisions: the path each vehicle takes and the order in which vehicles occupy each segment. One vehicle's yielding then delays another vehicle's arrival and, in turn, downstream operations and their transport occupancies. Studies that build such schedules before execution have shown that corridor contention changes which machine is preferable, so candidate schedules must be evaluated with time-window routing on a reservation table [9, 17, 20]. These works answer how to build a conflict-free schedule when information is complete. Once the shop has executed up to time t_{now} and a disturbance occurs, the questions become different: which corridor occupancies are invalidated, which must be rewritten to restore feasibility, and whether recovery can be completed within milliseconds without restarting a population-based search.

The gap. In dynamic scheduling of flexible job shops with transportation, the rescheduling scope is usually delineated on the operation-precedence graph: operations that can no longer be executed as planned serve as seeds, and their job successors and machine successors are added, possibly up to θ steps. Two dynamic scheduling studies with AGV transport delineate the scope in this way [19, 24], a practice that traces back to affected-operation rescheduling (AOR) [1, 16]. This is reasonable when the disturbance is an operation event, such as a machine failure. When the schedule records corridor occupancies with time windows, however, a *corridor blockage* closes a corridor during $[t_a, t_b)$ without modifying any machine assignment or processing order. No operation is directly affected, so the scope is empty and the method concludes that no rescheduling is needed. Yet the planned occupancies that overlap the blockage interval are no longer feasible, and a vehicle that waits or detours blocks subsequent vehicles through yielding relations, so the invalidation propagates across the reservation table. By definition, the operation-precedence graph contains no yielding relations among corridor occupancies and therefore cannot express this propagation; the gap arises only in settings that record corridor occupancies with time windows. We accordingly distinguish **Class A** disturbances, which make some operations unexecutable (robotic-arm failure, processing-time deviation, rush orders, vehicle failure), from **Class B** disturbances, which act only on the reservation table (corridor blockage, corridor slowdown); Class B is the counterexample to operation-level scoping (Section 3.3). Figure 1 illustrates the gap.

Stance. The reservation impact set is *a measurement, not a decision*. We do not use yielding relations to steer the search toward better assignments or paths, which would be schedule improvement whose gains are easily offset when another constraint becomes the bottleneck. We use them only to delineate which occupancies are already invalid; once the edges are fixed, the result is unique. The impact set is not necessarily the minimal set needed to restore feasibility, because the minimal set depends on the permitted repair actions (Remark 2). Its role also differs from that of global rescheduling: it answers whether feasibility can be restored *quickly* while confining rewrites to the set, not whether a better makespan can be found. Confining rewrites matters because every rewritten occupancy is a route or time window that must be reissued to a vehicle already executing the plan; keeping the rest unchanged is a form of schedule stability [21].

Contributions.

C1. A class of invalidation the operation-precedence graph cannot capture. Under corridor blockage and slowdown, operation-level scoping returns an empty scope although the original schedule is infeasible (50/50,

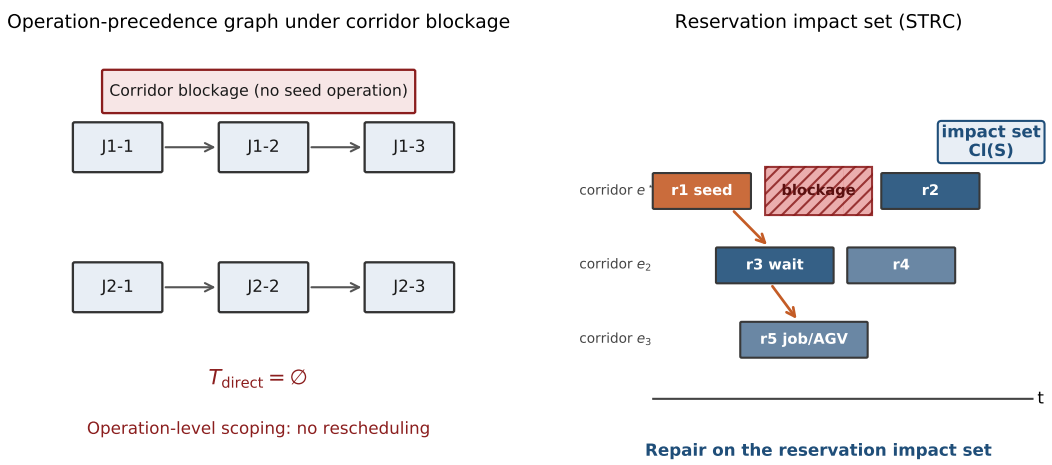


Fig. 1. Motivating example. Left: a corridor blockage makes no operation unexecutable, so the operation-precedence graph contains no seed operation. Right: the corridor occupancies that overlap the blockage interval become seeds and grow into a reservation impact set along yielding and succession relations.

Section 6.2); the root cause is that the operation-precedence graph has no yielding relations among occupancies (Proposition 1).

C2. The reservation impact set as the rescheduling scope. STRC delineates the scope by closure over yielding and same-vehicle, same-job, and same-machine succession. Membership is uniquely determined, no dependency edge leaves the set (50/50), and under Class A disturbances the set contains every occupancy already included by the operation-precedence graph (Section 6.6).

C3. A matching local recovery with the sources of each result separated. Under the same network, router, and checker, replacing only the scope object raises feasibility from 0/50 to 50/50 (Section 6.2). Bounded repair responds within milliseconds but has a worse makespan than global rescheduling, with no crossover as the budget grows (Section 6.5); against rewriting all unfinished occupancies, only the reduction in rewrites can be attributed to the impact set (Section 6.7).

Integrated scheduling before execution uses the reservation table to build a conflict-free plan; this paper uses the same table at execution time to locate the invalidated scope.

Section 2 reviews related work; Section 3 states the setting and the recovery problem; Section 4 defines STRC and its properties; Section 5 describes bounded repair; Section 6 reports the experiments; Section 7 concludes.

2 Related work

A basic question in dynamic scheduling is on which object the rescheduling scope is delineated after a disturbance. We review four classes of objects and summarize them in Table 2.

2.1 Local rescheduling in job shops

For operation-level events, delineating the scope by the affected operations is still common in dynamic scheduling with AGVs. Zhang et al. [24] reschedule the operations not yet processed after a machine breakdown; Tang et al. [19] classify disrupted operations and delivery tasks as retained, continued, or reconstructed, and compare full rescheduling, partial rescheduling, and right shift. This practice dates back to AOR [1, 16], which reschedules only the operations directly or indirectly affected. Delineating the scope along the time axis and enlarging it after failure have also been studied, including match-up [2] and Local Rescheduling with incrementally enlarged time windows [8]; right shift instead postpones the entire unfinished part. Rolling-horizon and decomposition methods trade off scale against quality [23, 26]; see [14, 21] for surveys. These works presuppose that the disturbance acts on operations; when it only makes a corridor impassable, the scope can be empty although the schedule is infeasible.

2.2 Occupancy order on exclusive resources

The alternative graph of Mascis and Pacciarelli [12] generalizes the disjunctive graph: vertices are occupations of block sections, fixed arcs encode route precedence, and paired alternative arcs encode which train passes first; D'Ariano et al. [4] use it for real-time train rescheduling. Our yielding relations express the same kind of precedence, but the use differs. In the alternative graph, occupancy order is a decision variable chosen to avoid positive cycles and optimize the objective; in our setting the order is already fixed, and we use it only to bring invalidated occupancies into the scope. Moreover, railways treat block sections as machines, so an unavailable section is itself an operation-level event and the mismatch studied here does not arise.

2.3 Local replanning in conflict-free multi-vehicle routing

In multi-agent path finding (MAPF), replanning only the affected agents is standard. Švancara et al. [18] compare Replan All, Replan Single, and Replan Affected, determining the affected agents by independence detection; Malka et al. [11] replan only the agents affected by observed map changes; lifelong MAPF treats spatiotemporal occupancy as a first-class constraint [25]. We adopt the mechanism "change only the affected, keep the rest". MAPF, however, has no job–operation–machine assignment, so operation-level scoping cannot miss a blockage there, and its affected set is the output of a detection algorithm that varies with the implementation, whereas the reservation impact set is fixed once the edges are given (Section 4.2).

2.4 Flexible job shop scheduling with transportation

Much recent work still takes travel times as a constant matrix [5, 13, 15, 22], in which a corridor blockage has no counterpart. Congestion-aware dispatching selects vehicles by proxies such as local density [7]. Van Os and Basten [20] formalize flexible job shop scheduling with conflict-free transportation and solve most benchmark instances to optimality; two-stage schemes first fix production and then resolve vehicle conflicts [17]; a learning-based scheduling layer can be coupled with a conflict-resolving routing layer [9]. These works build a conflict-free schedule *before* execution. Among the cited works, a reservation table and execution-time disturbances never appear together: Zhang et al. [24] state that "the collision of AGVs are ignored", while Sun et al. [17] resolve conflicts with time-window Dijkstra but assume that "AGVs are always available" and leave breakdowns to future research. When both are present, the only directly applicable practice is operation-level scoping, which yields an empty scope for a corridor blockage (50/50, Section 6.2). Table 1 lists the original sentences behind these paraphrases.

Table 1. The five paraphrases on which we base the gap, with the cited original text (English sentences quoted verbatim).

Our paraphrase	Cited original text	Location in source
Zhang et al. [24] take as the recovery scope the operations not yet processed after the breakdown	*"Rearrange the unprocessed operations of current scheme considering the limit of machine breakdown"*	Rescheduling procedure
Zhang et al. [24] state in their modeling assumptions that collisions between AGVs are ignored	*"The collision of AGVs are ignored."*	Modeling assumption (7)
Tang et al. [19] classify operations and delivery tasks by disturbance impact as retained, continued, or reconstructed	*"an event-driven partial rescheduling strategy is proposed, in which the disrupted operations and delivery tasks are classified into three categories: retained, continued, and reconstructed"*	Abstract
Van Os and Basten [20] obtain provable optimality on most benchmark instances; for a few, optimality cannot be guaranteed because of the time limit	*"For most benchmark instances, our LBBD-based approach finds optimal makespan values. Because of time boxing, only in a few cases, the optimality of the found solution cannot be guaranteed."*	Abstract
Sun et al. [17] assume vehicles are always available, ignore failures and charging, and list equipment maintenance and breakdowns as future research	*"It does not consider the charging problem and fault problem of the AGVs, and AGVs are always available"*; *"Breakdowns and charging problems are not taken into account."*; *"Future research will investigate the impact of equipment maintenance and breakdowns on production scheduling"*	Assumption list; future work

Table 2. Related work classified by *scope object*, i.e., the object on which "what must be rewritten" is defined after a disturbance.

Class	Representative work	Scope delineated on	Operation-precedence graph?	Relation to this paper
Local rescheduling in job shops	AOR [1, 16]; match-up [2]; LRS [8]; dynamic scheduling with AGVs [19, 24]	Operations / time windows	Yes	Operation-level scoping and right shift are reproduced as baselines
Occupancy order on exclusive resources	alternative graph [12]; real-time railway rescheduling [4]	Block-section occupancies	Yes, on the same graph as occupancies	Same kind as yielding relations; railways choose the order, we use the established order to delineate the scope
Conflict-free multi-vehicle routing	Online MAPF [11, 18]; LMAPF [25]	Affected vehicles	No	Rerouting inside the set and keeping the rest is already standard
Flexible job shop with transportation	constant matrix [5, 13, 15, 22]; integrated scheduling [9, 17, 20]	Set of operations	Yes; reservation table and execution-time disturbances never appear together	**Position of this paper**

3 The corridor-reservation setting and the execution-time recovery problem

3.1 System setting and notation

The setting follows the flexible job shop with conflict-free transportation constraints [20]. A feasible schedule $\mathcal{S}^0$ is a given input; how it was found does not affect how we delineate the occupancies to be rewritten.

3.1.1 Shop floor and four groups of decisions. The shop routing graph $G = (V, E)$ is strongly connected. Each machine m processes through a robotic arm at node $\mathrm{loc}(m)$, so a robotic-arm failure means a failure of that machine; $v_0 \in V$ is the

Table 3. Frequently used notation

Symbol	Meaning
$G = (V, E)$, τ_e	Corridor network; traversal time of corridor e
x, π, y	Machine assignment, processing sequence, vehicle assignment
R, $r = (e, [t^s, t^e), k, u)$	Reservation table; a half-open occupancy (vehicle k, leg label u)
G_T / G_R	Operation-precedence graph / reservation dependency graph
$\mathcal{S}^0$, $\mathcal{S}'$	Schedule before the disturbance, schedule after recovery
$\mathcal{D}$, t_{now}	Disturbance event; decision time
T_{direct}, T_{impact}	Directly disturbed operations / θ-step scope on G_T
$\text{Src}(\mathcal{D})$, U, R°	Seed occupancies; rewritable set; unfinished occupancies
$\text{Cl}(\cdot)$	Reservation impact set (obtained by STRC)
φ	Fraction of disturbed future operations (magnitude sweep)

load/unload station. There are n jobs, a machine set M, and a homogeneous fleet of unit-load vehicles $K = \{1, \dots, N_A\}$. Job j has a fixed chain $O_{j1} \to \cdots \to O_{jn_j}$; operation O_{ji} has candidate machines $\Omega_{ji} \subseteq M$ and processing time t^P_{jim} on machine m. A schedule fixes (i) the machine assignment x; (ii) the processing sequence π on each machine; (iii) the vehicle assignment y of each transport; and (iv) a conflict-free path for each vehicle move. We write $\mathcal{S} = (x, \pi, y, R)$, where the reservation table R records group (iv). Transports are derived from the assignment: a transport arises when two consecutive operations are at different nodes, and it is split into an empty leg and a loaded leg, each with its own task label u. Loading and unloading times are zero. Rewritability, keeping, and rerouting are all decided at the granularity of legs, not whole operations.

3.1.2 Corridor reservation and conflict freedom. Each corridor $e \in E$ is an exclusive resource with traversal time τ_e, shared by both directions, which rules out head-on conflicts and overtaking. Occupancies are half-open intervals, so a second vehicle may enter exactly at the right endpoint; nodes have ample parking capacity, so waiting at nodes is always allowed. A reservation record $r = (e,\ [t^s, t^e),\ k,\ u) \in R$ means that vehicle k occupies e during $[t^s, t^e)$ for leg u, and distinct occupancies of the same corridor must be disjoint. Time-window routing is the primitive shared by our method and all baselines: given origin, destination, earliest departure, and the current R, it returns a path disjoint from existing occupancies. We implement it as time-window Dijkstra [17] and change only *which* legs are sent to it. Unlike van Os, who reserves nodes, and Lyu, who additionally imposes node capacities [10, 20], we reserve edges.

3.1.3 Operation-precedence graph and reservation dependency graph. The operation-precedence graph $G_T = (T_T, E_T)$ has operations as vertices and job and same-machine precedence as edges; it does *not* contain yielding relations between occupancies. The reservation dependency graph $G_R = (R, E_R)$ has occupancies as vertices and the dependencies induced by yielding, same vehicle, same job, and same machine as edges (Definition 3). The vertex sets are disjoint, one representing processing and the other traversal; hence a disturbance that changes only corridor time windows yields no seed on G_T but a nonempty seed set on G_R (Figure 2; notation in Table 3).

3.2 Execution-time assumptions

A1–A4 are shared by all compared methods, so that differences can be attributed to the scope rather than the model; A5 constrains only our method.

A1. Feasible initial schedule. $\mathcal{S}^0$ passes the conflict-freedom check before the disturbance.

A2. No backtracking on finished occupancies. Occupancies with $t^e \le t_{\text{now}}$ are not rewritable.

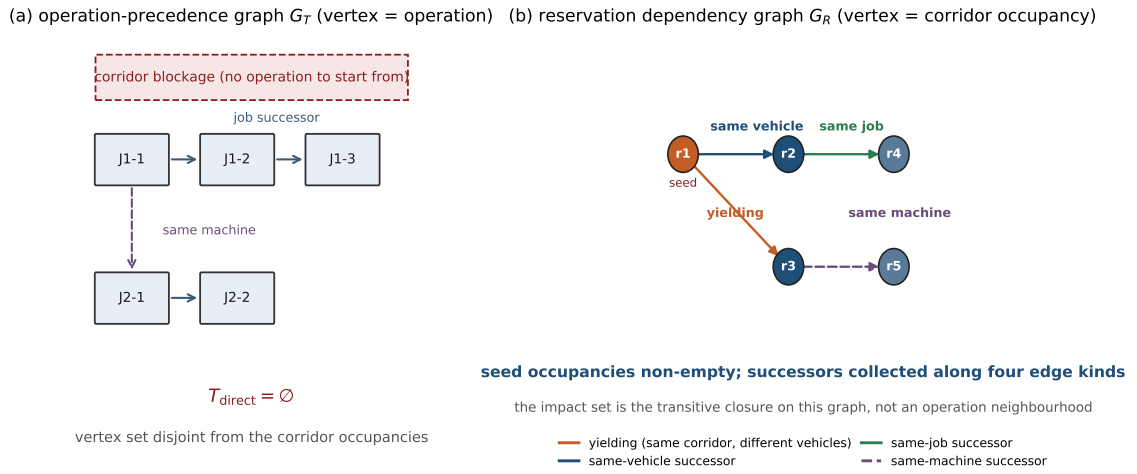


Fig. 2. The two graphs have disjoint vertex sets. (a) Operation-precedence graph G_T: a corridor blockage finds no seed operation. (b) Reservation dependency graph G_R: starting from the occupancies overlapping the blockage window, subsequent occupancies are included along yielding, same-vehicle, same-job, and same-machine edges; the reached set is $\mathrm{Cl}(\mathrm{Src})$, and occupancies outside it are kept as planned.

A3. Single, fully observed disturbance. Disturbance streams, prediction errors, and rolling responses are out of scope.

A4. Exclusive corridors, ample buffers, homogeneous unit-load fleet. A blockage is injected into the routing layer as a no-entry window; finite buffers, charging, acceleration, and loading durations are not modeled.

A5. Permitted recovery actions. For Class B, x and π are kept and only rerouting is performed, with scope escalation upon failure. Class A additionally allows reassigning a failed vehicle's tasks to another vehicle, or a failed machine's unfinished operations to other feasible machines. π is never changed and the search is not restarted; the global-rescheduling baseline may change x and π (Section 5.3).

3.3 A dichotomy of disturbances

Definition 1 (Class A/B disturbances). *A disturbance is of **Class A** if it directly invalidates the assignment or* executability *of some operations (robotic-arm failure, processing-time deviation, rush-order insertion, vehicle failure). It is of **Class B** if it only makes some corridor time windows unavailable without making any operation inexecutable (corridor blockage, corridor slowdown).*

Remark 1 (Why vehicle failure is Class A). *A vehicle failure changes no machine assignment, but the operations the vehicle transports cannot start as planned. Through the mapping "vehicle→operations it transports", G_T still contains seed operations. This holds only if the method maintains a vehicle–task mapping; Section 6.6 reports results under this convention.*

Definition 2 (Scopes on the operation-precedence graph and seed occupancies). *$T_{\text{direct}}(\mathcal{D})$ is the set of operations that cannot be executed as planned; for Class B, $T_{\text{direct}} = \varnothing$ by convention. $T_{\text{impact}}(\mathcal{D}, \theta)$ collects the operations reached from T_{direct} on G_T along job and same-machine successor edges within depth θ. The seed set $\mathrm{Src}(\mathcal{D}) \subseteq R$ contains*

only occupancies with $r.t^e > t_{\text{now}}$ and is given by disturbance type: (a) ***corridor blockage/slowdown*** *$\mathcal{D} = (e, [t_a, t_b))$: $\{ r : r.e = e,\ [r.t^s, r.t^e) \cap [t_a, t_b) \neq \varnothing \}$ (for multiple windows, the union); blockage and slowdown share this rule and differ only in rerouting, where a blockage is a no-entry window and a slowdown scales traversal progress inside the window; (b)* ***vehicle failure*** *$\mathcal{D} = (k)$: $\{ r : r.k = k \}$; (c)* ***robotic-arm failure*** *$\mathcal{D} = (m, F)$: for each job, all unfinished occupancies from its smallest operation index in F onward, so that the seeds are not shorter than the affected set on G_T when the inbound transport has already finished.*

Proposition 1 (The operation-precedence graph yields an empty scope for Class B). *Let $\mathcal{D}$ be a corridor blockage. Then $T_{\text{impact}}(\mathcal{D}, \theta) = \varnothing$ for every finite $\theta \geq 0$. If, in addition, $\text{Src}(\mathcal{D}) \neq \varnothing$, then $\mathcal{S}^0$ is infeasible under $\mathcal{D}$.*

Proof. Since the blockage window is finite and buffers are ample (A4), vehicles can traverse outside the window, so no operation becomes inexecutable and $\mathcal{D}$ is of Class B; hence $T_{\text{direct}} = \varnothing$ and the θ-step expansion is empty. If $r \in \text{Src}(\mathcal{D})$ exists, r overlaps the no-entry window on the same exclusive corridor, which violates conflict freedom (A4); hence $\mathcal{S}^0$ is infeasible. □

The first part follows from the convention; the nontrivial part is the second. Hence "an empty scope on $G_T \Rightarrow$ no rescheduling needed" fails when corridors are reserved by time windows, independently of any repair algorithm.

3.4 The recovery problem

Problem 1 (Bounded rescheduling with the scope delineated on the reservation table). *Given $\mathcal{S}^0$, $\mathcal{D}$, and t_{now}, find $\mathcal{S}'$ such that (i) $\mathcal{S}'$ is conflict-free under $\mathcal{D}$; (ii) occupancies outside the rewrite scope keep their corridors, intervals, and vehicles as in $\mathcal{S}^0$ as far as possible; (iii) the runtime is subject to a budget. The secondary objective is the makespan $C_{\max}(\mathcal{S}')$.*

Bounded rescheduling first delineates the rewritable set $U \subseteq R$ and then reroutes only the transports in U; we call the second step *bounded repair* (Section 5). We take $U = \text{Cl}(\text{Src})$; the baseline instead maps T_{impact} to all occupancies whose task label lies in it and whose $t^e > t_{\text{now}}$, which is empty under Class B by Proposition 1. The two are denoted R2 and R1 in the experiments.

4 The spatiotemporal reservation closure

4.1 Reservation dependency edges

Definition 3 (Reservation dependency graph). *The vertices are the occupancies in R; an edge $r \to r'$ means "invalidating r may force r' to be rerouted". There are four kinds of edges: (1)* ***yield edges*** *between occupancies of the same corridor by different vehicles whose intervals overlap* or abut *($r.t^e = r'.t^s$), from the earlier entrant to the later one—abutting is legal, but if the former is delayed the latter must give way; (2)* ***same-vehicle successors****, adjacent in time order; (3)* ***same-job successors****, from the occupancies of operation i to those of operation $i+1$; (4)* ***same-machine successors****, between the occupancies induced by two operations adjacent on one machine. This yields $G_R = (R, E_R)$. The edge set is a modeling choice; its completeness is discussed in Remark 2.*

Definition 4 (STRC). *Given seeds $S \subseteq R$, a horizon H, and t_{now}, let $R^\circ = \{ r \in R : r.t^e > t_{\text{now}},\ r.t^s < H \}$ and let G_R° be the subgraph induced by R°. The reservation impact set is*

$$\text{Cl}(S) = \left\{ v \in R^\circ : \exists\, s \in S \cap R^\circ \text{ such that } s \to^* v \text{ in } G_R^\circ \right\},$$

i.e., all unfinished occupancies reachable from the unfinished seeds, seeds included. We use $H = C_{\max}(\mathcal{S}^0) + 1$, *so the horizon filters nothing and is kept only for rolling horizons. With adjacency lists,* $\mathrm{Cl}(S)$ *is computed by one breadth-first pass in* $O(|R^\circ| + |E_R^\circ|)$ *time.*

4.2 Containment

Proposition 2 (Structural closure). *For any* $S \subseteq R$, G_R° *contains no edge* $u \to v$ *with* $u \in \mathrm{Cl}(S)$ *and* $v \in R^\circ \setminus \mathrm{Cl}(S)$.

Proof. If such an edge existed, then $s \to^* u \to v$ for some unfinished seed s, so $v \in \mathrm{Cl}(S)$, a contradiction. □

Closure is thus a corollary of the definition once the edge set is fixed; its physical completeness depends on the choice of edges.

Remark 2 (The reservation impact set and the minimal rewritable set). *Closure neither guarantees that the impact set suffices to restore feasibility nor that it is small. First, a* minimal *rewritable set is defined only relative to a repair capability (rerouting only, or also reassignment), whereas* Cl *is determined by* G_R *alone and cannot be both. Second, the edges over-include: a yield edge is created for any overlap without checking whether the later vehicle could detour, and same-vehicle edges may pull in chain tails the delay never reaches. In the opposite direction, the known omission—abutting intervals—has been folded into the yield edges; a difference probe finds 0 leaks (0/50 pairs), which tests this omission but does not prove completeness. Third, empirically the impact set covers about 92% of the unfinished occupancies (Section 6.6), less than one tenth below the trivial upper bound of rewriting all of them. The savings of bounded repair therefore come mainly from* not rewriting finished occupancies *and* not restarting the search*; "bounded" does not mean "small". Section 6.7 nevertheless shows that this small difference in set size lowers the rewrite fraction significantly. The impact-set share does not rise with instance size (Section 6.9), and static network indicators fail to predict it (Section 6.4).*

Proposition 3 (A sufficient condition for unchanged fields outside the set). *Let* $U = \mathrm{Cl}(S)$, *and let level-1 rerouting (Section 5.1) satisfy: (i) the no-entry windows are written into the routing layer; (ii) for every* $r \in R^\circ \setminus U$, *its fields* $(e, [t^s, t^e), k)$ *are pre-committed to the new table and the pre-commit succeeds; (iii) the router generates new occupancies only for transports in* U*; (iv) the final schedule passes the conflict-free check. Then the corridor, interval, and vehicle of every* $r \in R^\circ \setminus U$ *in the recovered table agree with* $\mathcal{S}^0$. *The quantifier ranges over occupancies, not tasks.*

Proof. By (ii), occupancies outside the set enter the new table with their original fields before the replay; by (iii), the replay neither deletes nor rewrites them; the check in (iv) only rules out conflicts and alters no committed field. □

Premises (i)–(iv) are execution conditions that can be checked against Algorithm 1; beyond them, the proposition relies on A1, A2, and A4. A failed pre-commit indicates an incomplete seed set. If scope escalation enlarges U, the proposition holds only for the occupancies finally left outside. Premise (iii) is easily violated in practice when rerouting is decided at a unit coarser than an occupancy; Section 6.2 reports such a case.

5 Bounded repair on the reservation impact set

Figure 3 outlines the procedure: delineate U by the reservation impact set, reroute only the transports inside it while keeping the rest as planned, and escalate U if the result is still infeasible. All methods share the same corridor network, time-window routing, and conflict-free check.

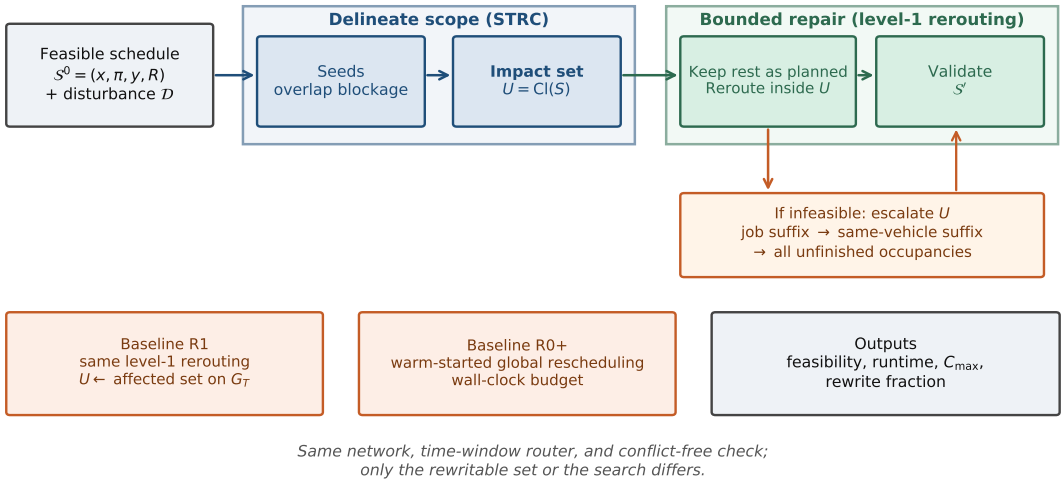


Fig. 3. Bounded repair scoped on the reservation impact set. Top row: the impact set is obtained from the seeds and vehicles are rerouted; middle row: the rewritable scope is escalated after failure; bottom row: the baseline scoping on the operation-precedence graph, the warm-started global rescheduling baseline, and the output metrics. Three further baselines operate within the millisecond range (Section 6.7).

Algorithm 1 Level-1 rerouting replay ReplayReroute

Require: schedule $\mathcal{S}^0 = (x, \pi, y, R)$, disturbance $\mathcal{D}$, rewritable set U, time t_{now}
Ensure: schedule $\mathcal{S}'$ or failure

write $\mathcal{D}$ into the routing layer ▷ blockage: no-entry window; slowdown: scaled τ
$R' \leftarrow \varnothing$
for each occupancy outside U, and each $r \in U$ with $r.t^e \le t_{\text{now}}$ **do**
    pre-commit its original fields $(e, [t^s, t^e), k)$ to R'; **if** it conflicts with a no-entry window **then return** failure
end for
for each $r \in U$, in the original processing order **do**
    route from the vehicle's node at t_{now} on the current R' and commit the new occupancies
end for
advance machine idle times and job ready times according to π
if independent check passes (no conflicts, fields outside U unchanged) **then return** $\mathcal{S}' = (x, \pi, y, R')$ **else return** failure

5.1 Level-1 rerouting

Given U, level-1 rerouting keeps the vehicle and machine assignments and replays in the original processing order (Algorithm 1). It operates at the granularity of corridor segments: segments traversed before t_{now} are pre-committed unchanged, and planning starts from the vehicle's node at that time. Delineating the scope takes one breadth-first pass, whereas the replay calls the router once per leg in U; the latter dominates the runtime. No population-based search is launched. Section 6.2 fixes this procedure and replaces only the source of U, so that differences in feasibility can be attributed to the scope object.

Algorithm 2 Bounded repair on the reservation impact set (level-1 rerouting + scope escalation)

Require: schedule $\mathcal{S}^0$, disturbance $\mathcal{D}$, time t_{now}
Ensure: schedule $\mathcal{S}'$ or failure
1: $U \leftarrow \text{Cl}(\text{Src}(\mathcal{D}))$
2: **for** round = 0, 1, 2, 3 **do**
3: $\quad \mathcal{S}' \leftarrow \textsc{ReplayReroute}(\mathcal{S}^0, \mathcal{D}, U, t_{\text{now}})$; **if** $\mathcal{S}'$ is feasible **then return** $\mathcal{S}'$
4: $\quad$ update U by the scope-escalation rule ▷ job suffix / same vehicle / all unfinished
5: **end for**
6: **return** failure

5.2 Scope escalation after failure

Escalating the scope has precedents: Local Rescheduling [8] relaxes the time window step by step, and AOR adds successor operations (Section 2.1); we escalate along reservation dependencies. When keeping the occupancies outside U is too restrictive, the replay may fail at a transport–operation hand-off; we then enlarge U and retry: (i) add the unfinished occupancies in the same-job suffixes of operations in U; (ii) add those in the same-vehicle suffixes of the vehicles involved; (iii) set $U \leftarrow R^\circ$. Each round remains a deterministic single pass (Algorithm 2).

5.3 Comparison with global rescheduling

The global rescheduling baseline R0+ is a genetic algorithm that encodes (x, π) as a chromosome, is warm-started from $\mathcal{S}^0$, and searches under the same no-entry constraints within a wall-clock budget. One *decoding* routes all transports from scratch on the table with the no-entry windows written in; finished occupancies are pre-committed, and only operations not yet started are scheduled from t_{now}. Because R0+ can reassign and resequence, it is more likely to shorten $C_{\max}$, but its runtime is set by the budget. Bounded repair is therefore not expected to win on makespan; its strength is runtime. To test whether scoping on the impact set matters *within* a millisecond runtime, Section 6.7 adds three millisecond baselines (RS, RD, RA), of which RA differs from our method only in which occupancies are rewritable.

6 Experiments

6.1 Protocol

Instances. The main batch has five instances: the small `example_3x3x2`, the congested dumbbell `congested_8x4x4`, and three generated `S8x4x4` layouts (`high`, `funnel`, `mid`) that share size and flexibility and differ only in corridor topology. Generator parameters and the generation seed (42) are encoded in the file names; the generation seed is independent of the run seeds. We use ten run seeds (42, 7, 2024, 99, 123, 13, 1, 777, 31415, 8), giving $5 \times 10 = 50$ instance–seed pairs ("cells"). All cell-wise comparisons and paired tests are aligned on the key (instance, seed), under which all methods read the same initial schedule, blockage, and decision time. The initial schedule is obtained by replaying the time-window router in processing order and passes the independent check.

Second batch with external layouts. To ensure that the layouts are not chosen by us, a second batch of 5 instances (50 pairs) transcribes the grid, station and machine placement, and missing edges from Appendix A of Lyu et al. [10], reconciled by script against the encoding shipped with the van Os toolkit. All other parameters are matched to the main batch, so the two batches differ only in layout source. We omit the 3-machine layout, which cannot give every operation two candidate machines at $f = 0.6$. The five remaining layouts lie on only 2 distinct networks (4×4 with 4 or 5 machines; 5×5 with 6, 7, or 8 machines), so along network geometry there are 2 independent samples. Per-edge

Table 4. Compared methods. All share the network, time-window router, initial schedule, and independent checker, and none rewrites finished occupancies.

Method	Rewritable scope	Recomputation	Termination
R1	Affected set on G_T ($\theta = 2$) mapped to occupancies	Level-1 rerouting	Single pass
R2 (STRC)	Reservation impact set Cl(Src)	Level-1 rerouting	Single pass
RA	All unfinished occupancies R°	Level-1 rerouting	Single pass
RS	None delineated; all unfinished occupancies shifted	Uniform right shift	Single pass
RD	None delineated	One fixed-prefix decoding	Single pass
R0+	None delineated	Warm-started GA (population 40)	Wall-clock budget

durations of these layouts were never published, so we use equal weights; the separate loading and unloading stations are merged into one; and their job data are not used, because in public instance families transport takes only a few percent of processing time and corridor contention would be unobservable. This batch therefore serves as a set of topologies not designed by us, not as a reproduction of published reference values.

Disturbances. Unless noted otherwise, $t_{\text{now}} = 0.35\, C_{\max}(\mathcal{S}^0)$. E1–E3, E5, and E7 block a busy corridor in a single window; E6 adds three further types; E4 blocks a corridor in the load/unload minimum cut; the magnitude sweep takes the future reservation windows of the first fraction φ of future operations as micro-blockages.

Metrics. An occupancy counts as *rewritten* if its corridor, interval, or vehicle after repair differs from $\mathcal{S}^0$; the *rewrite fraction* is the share of rewritten occupancies in the whole table R. It differs from the *rewritable share* $|U|/|R^\circ|$, which counts the occupancies a method is allowed to change. Makespan *degradation* is $(C_{\max}(\mathcal{S}') - C_{\max}(\mathcal{S}^0))/C_{\max}(\mathcal{S}^0)$. A *finished-occupancy violation* is a rewritten occupancy with $t^e \le t_{\text{now}}$, which A2 forbids.

Methods and conventions. Table 4 lists the methods. R1 transplants AOR scoping [1, 16] into our reservation setting: the θ-hop truncation and the mapping from operations to reservations are ours, since the original setting has no reservation table; its results reflect the rule in this setting, not the original method in its own. R1/R2 differ only in the scope *object*, RA/R2 only in *whether* the scope is restricted, and RD/R0+ only in how long they search. Only R0+ terminates on a budget; all others are deterministic single passes, so the comparison among them in Section 6.7 is the like-for-like one. Scope escalation is disabled in E3, E5, and E7, so that a fallback to all unfinished occupancies cannot mask the difference between scope objects; its end point is RA, measured separately. It is enabled in the magnitude sweep. The main batch establishes the phenomena under full control of the instance parameters; the second batch only tests whether they depend on our layouts, and is pooled with the main batch only in E5.

Implementation and statistics. All code is pure Python 3.7 (standard library only), run single-threaded on an Intel Xeon W-1270 with 64 GB RAM under Windows 10; runtime conclusions are stated as ratios or orders of magnitude. Code, instances, disturbance inputs, cell-wise CSV results, and the scripts generating every table and figure are available as an anonymous repository during review and will be archived with a DOI upon acceptance. The paper contains two inferential tests, both in Section 6.7: R2 versus RA on rewrite fraction (primary) and R2 versus RD on makespan, each a two-sided Wilcoxon signed-rank test paired on the cell key; Holm correction does not change either conclusion. For the primary test, the Hodges–Lehmann estimate is 0.037 (bootstrap 95% CI $[0.018,\ 0.074]$, 10^4 resamples) and the mean paired difference is 0.064 (CI $[0.039,\ 0.093]$). All other counts, means, and medians are descriptive.

Table 5. E1–E3 per instance (means over 10 seeds). $|R|$: occupancies in the whole table; $\mathrm{Cl}/|R|$: share of the reservation impact set. On every instance, all checks of E1, E2a, E2b and R2 feasibility pass in 10/10 cells and R1 is feasible in 0/10; totals are given in the text.

Instance	Mean $\lvert R\rvert$	Mean $\lvert\mathrm{Src}\rvert$	Mean $\lvert\mathrm{Cl}\rvert$	$\mathrm{Cl}/\lvert R\rvert$	R2 ms
`example_3x3x2`	28.6	5.6	16.9	0.59	0.66
`congested_8x4x4`	85.0	15.0	48.4	0.57	2.84
`S8x4x4 (high)`	150.5	15.5	88.3	0.59	3.40
`S8x4x4 (funnel)`	149.1	15.5	85.4	0.57	5.09
`S8x4x4 (mid)`	147.6	15.0	82.8	0.56	3.96

6.2 E1–E3: empty scope, containment, and boundary ablation

Table 5 summarizes the per-instance statistics. Over the 50 cells: (E1) the three conditions of Proposition 1—$T_{\mathrm{impact}} = \varnothing$, a nonempty seed set, and an infeasible original schedule—hold together in 50/50 cells; the first follows from the Class B convention, and the substantive result is that the original schedule is infeasible every time. (E2a) The impact set is closed under outgoing edges in 50/50 cells, confirming that the implementation matches Proposition 2. (E2b) After level-1 repair, the corridor, interval, and vehicle of every occupancy outside the set are unchanged in 50/50 cells, i.e., the premises of Proposition 3 hold under our protocol. (E3) With level-1 rerouting fixed and escalation disabled, R1 is feasible in 0/50 cells and R2 in 50/50. The two arms share the rerouting procedure, initial schedule, decision time, and blockage; **the only difference is which occupancies are rewritable**. R1's rewritable set is empty, so the replay leaves the schedule unchanged and it still occupies the blocked corridor: R1 fails because it decides that no rescheduling is needed, not because of repair quality. The median runtime of level-1 rerouting is 3.2 ms (maximum 12.8 ms).

$|\mathrm{Cl}|$ grows with the instance, while $\mathrm{Cl}/|R|$ stays within 0.56–0.59 (pooled median 0.571, per-instance IQR 0.024–0.049) and does not distinguish the three `S8x4x4` layouts; between-instance variation is of the same order as between-seed variation. The contrast does not depend on equal edge weights: under three non-uniform weightings (`lognormal`, `lu_far`, `split`), E1 is unchanged, R1 remains infeasible everywhere, and R2 is level-1 feasible in 50/50, 50/50, and 49/50 cells.

An implementation defect that mimics an edge-set defect. When rerouting was decided per *operation*, E2b passed in only 24/50 cells, which is easily misread as missing dependency edges. All 84 drifting occupancies were empty legs replanned together with a loaded leg in the impact set, i.e., premise (iii) of Proposition 3 was violated by the implementation, not by the edge set. Deciding per leg removed this drift, and a separate audit of finished occupancies then revealed 86 rewrites in 36 cells caused by legs spanning t_{now}; deciding per segment reduced the audit to zero, left all pass rates unchanged, and improved $C_{\max}$ in 6 cells. All reported results use the segment-granularity implementation.

6.3 Case Gantt chart

Figure 4 shows one corridor blockage on `example_3x3x2` (seed 42) under the E3 protocol: $T_{\mathrm{impact}} = \varnothing$, whereas $|\mathrm{Src}| = 6$ and $|\mathrm{Cl}| = 18$. After level-1 repair, the operations whose transports lie in the impact set are rearranged, the others stay in place, and the schedule regains feasibility in under a millisecond ($C_{\max} : 57 \longrightarrow 86$). The figure illustrates the form of the local change and is not used to compare makespans.

6.4 E4: static structure does not predict the impact-set size (exploratory)

We test whether the impact-set share can be predicted from the network alone, using the load/unload minimum cut (LU cut), the funnel share, and corridors per node, while blocking a corridor in the LU cut. On the in-house layouts the

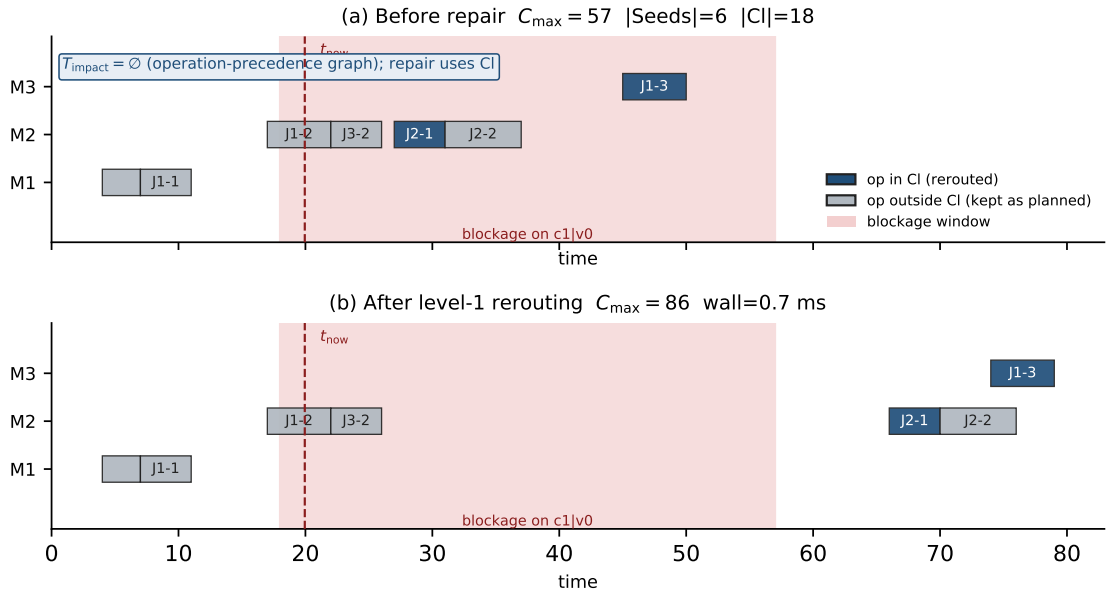


Fig. 4. Case Gantt chart (`example_3x3x2`, seed 42). (a) Original schedule; blue operations have transports in Cl, gray ones are kept; the dashed line marks t_{now} and the band the blockage window. (b) After level-1 repair.

answer is inconclusive: in a heavier setting (12 jobs, 8 machines, 12 vehicles, five seeds), `funnel` has a higher median share than `high` (0.254 vs. 0.204), but the per-seed ranges overlap ($[0.180, 0.309]$ vs. $[0.156, 0.372]$) and one seed reverses the order. Since these layouts differ little in geometry, "the indicator is too coarse" cannot be told apart from "the instances are too similar". The external layouts separate the two (Table 6). On the same 4×4 network, moving from 4 to 5 machines changes the median share by 0.207; on the same 5×5 network, three machine configurations span 0.075; the three in-house networks span only 0.051. Meanwhile the LU cut is always 2 and the funnel share always 0 on the external batch (stations at grid corners have out-degree 2), and corridors per node takes only two values. **A constant indicator cannot explain a varying quantity**: static cuts do not determine the impact-set size, and the machine-configuration dimension cannot be ignored. Because the in-house side varies only the network, this does not show that machine configuration matters more than geometry. The result is negative and supports none of the contributions; Section 7.1 points to schedule-dependent candidates.

6.5 E5: trade-off against global rescheduling

We test whether a *crossover* exists, i.e., whether R0+ overtakes bounded repair in C_{max} only once the budget is relaxed. R2 is a single deterministic computation, so its C_{max} does not vary with the budget; with the seed fixed, a larger budget lets R0+ run the same trajectory for more generations, so its best-so-far C_{max} is non-increasing. Hence, once the order is settled at the *smallest* budget, no larger budget can reverse it. At 0.2 s, R0+ is strictly better in 11 of the 12 cells, ties in 1, and is worse in none; over all 36 budget points the win/tie/loss is 35/1/0 (both batches pooled; Table 7 and Figure 5 show the in-house half).

Table 6. E4: in-house versus external layouts (blocking a corridor in the LU cut, ten seeds; median Cl/$|R|$). Rows are matched parameter by parameter. External instances lie on 2 networks (N_1: 4×4; N_2: 5×5); rows within a group share the network. Ranges are within groups.

Group	Instance	Machines	Nodes/corridors	LU cut	Funnel	Cl/$\|R\|$	Range
External N_1	`LyuL2`	4	16/23	2	0	0.512	
External N_1	`LyuL3`	5	16/23	2	0	0.305	0.207
External N_2	`LyuL4`	6	25/38	2	0	0.385	
External N_2	`LyuL5`	7	25/38	2	0	0.422	
External N_2	`LyuL6`	8	25/38	2	0	0.460	0.075
In-house	`high`	4	10/10	2	0.286	0.497	
In-house	`funnel`	4	9/8	1	0.286	0.491	
In-house	`mid`	4	11/12	2	0.286	0.446	0.051

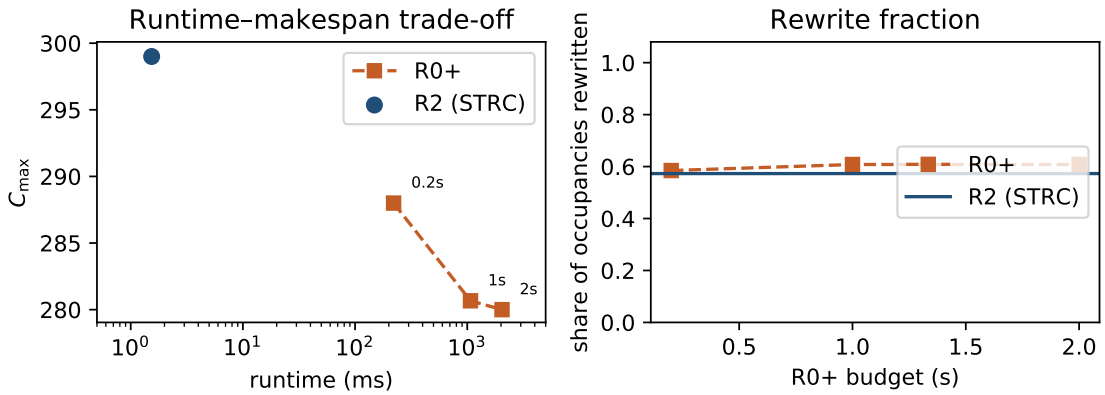


Fig. 5. E5: budget sweep of R0+ (which does not rewrite finished occupancies) versus bounded repair (`congested_8x4x4`, mean over 3 random seeds). Left: runtime versus C_{max}; the two curves do not intersect. Right: occupancy rewrite fraction, about 0.6 for both methods.

Table 7. E5: warm-started global rescheduling R0+ versus bounded repair R2 (in-house layouts, mean over seeds $\{42, 7, 2024\}$). R2 does not depend on the budget.

Instance	Budget/s	R0+ C_{max}	R2 C_{max}	Rewrite fraction R0+ / R2
`congested_8x4x4`	0.2	279.3	299.0	0.61 / 0.57
	1.0	276.0	299.0	0.61 / 0.57
	2.0	275.7	299.0	0.61 / 0.57
`example_3x3x2`	0.2–2.0	78.3	88.7	0.60 / 0.60

Below 0.2 s the order may reverse: evaluating the initial population alone takes about as long as the smallest budget (one decoding takes 4.16 ms), whereas R2 needs 2.74 ms. Neither R0+ nor R2 rewrites finished occupancies (0 violations at every budget point), so the makespan gap comes from global reassignment and resequencing, not from backtracking. Bounded repair is two to three orders of magnitude faster, and its rewrite fraction is of the same order (50%–66%). As Section 6.7 shows, this speed comes from single-pass rerouting, not from the impact set.

Table 8. E6: disturbance type × scope object (50 pairs per type). $|R^\circ|$: unfinished occupancies. "R1 empty": cells where scoping on G_T deems no rescheduling necessary. The last column uses no scope escalation.

Disturbance	Class	R1 empty	Mean $\lvert U_{R1}\rvert$	Mean |Cl|	Cl/$\lvert R^\circ\rvert$	R2 feasible
Corridor blockage	B	50/50	0.0	64.4	0.92	50/50
Corridor slowdown	B	50/50	0.0	64.4	0.92	50/50
Vehicle failure	A	0/50	31.1	60.6	0.85	50/50
Robotic-arm failure	A	0/50	42.8	65.9	0.94	50/50

Table 9. E7: methods within the millisecond range (50 pairs, E3 protocol, all methods feasible in every cell). "Violations": cells with rewritten finished occupancies; "Rewritable": share of unfinished occupancies included in the rewrite; last column: $C_{\max}$ win/tie/loss against R2.

Method	Violations	ms (median)	Mean degradation	Rewrite fraction	Rewritable	W/T/L vs. R2
RS right shift	0/50	1.42	58.8%	0.627	—	7/16/27
RD re-decoding	0/50	4.16	49.0%	0.618	—	22/14/14
RA all unfinished	0/50	2.76	49.9%	0.604	1.00	0/46/4
R2 bounded repair	0/50	2.74	49.8%	0.540	0.92	—

6.6 E6: disturbance types crossed with scope objects

On the same 50 pairs we add corridor slowdown (progress scaled by $\tau_{\text{mult}} = 2$ inside the window), vehicle failure, and robotic-arm failure (Table 8). **First**, the A/B dichotomy holds without exception: scoping on G_T is empty for both Class B types and nonempty for both Class A types in every cell. Blockage and slowdown share the seed rule and hence give identical impact sets, so there are 150 independent scoping measurements, not 200. **Second**, over all 100 Class A measurements $U_{R1} \subseteq$ Cl, strictly in 87/100 cells: switching to the reservation impact set never misses an occupancy already included by G_T. For robotic-arm failure this holds by construction (the seed rule takes the whole job suffix, while R1 truncates at $\theta = 2$); for vehicle failure it is an empirical result. The price is a larger rewritable set (31.1 $\rightarrow$ 60.6 and 42.8 $\rightarrow$ 65.9). **Third**, all four types are repaired in every cell without escalation, Class A by vehicle swapping or machine reassignment. **Fourth**, under blockage the impact set covers about 92% of the unfinished occupancies, which is consistent with the Cl/$|R|$ of 0.56–0.59 in Table 5, because roughly half of the table is finished before t_{now}.

6.7 E7: comparisons within the millisecond range

E5 leaves open how much of the speed advantage is due to the impact set and how much merely to a slow baseline. We therefore compare R2 with three millisecond baselines (Table 4). **RS** postpones all unfinished occupancies by the smallest delay that clears the blocked corridor; because the shift is uniform, all "$\geq$" constraints remain satisfied without rerouting. **RD** keeps the chromosome and runs one fixed-prefix decoding, the fastest global recomputation in our implementation. **RA** makes all unfinished occupancies rewritable and runs the *identical* replay as R2; it is the only baseline that isolates the effect of the impact set.

The only faster feasible method is the right shift, and it is worse on both makespan and rewriting. RS needs 1.42 ms, about half of R2, but degrades the makespan by 58.8% versus 49.8% (R2 win/tie/loss 27/16/7) and rewrites more (0.627 vs. 0.540). Making the shift selective would first require determining which occupancies are affected—precisely the scope delineated here. (R1 is faster still, but feasible in only 0/50 cells.)

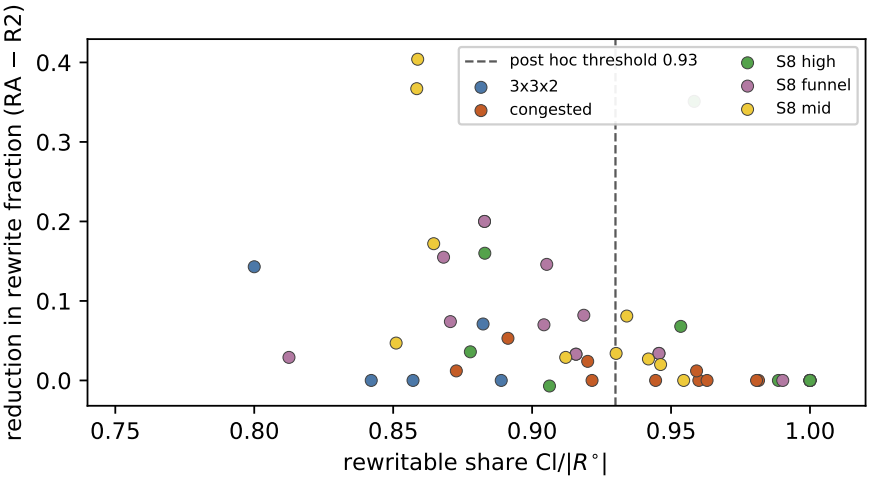


Fig. 6. E7 post hoc rule: rewritable share $\mathrm{Cl}/|R^{\circ}|$ versus the reduction in rewrite fraction of R2 relative to RA (50 pairs). The dashed line marks the in-sample threshold 0.93.

Even the fastest global recomputation is slower. RD takes 4.16 ms, 1.5 times R2, and any decoding-based search needs at least one decoding per candidate; the speed gap in E5 is therefore not an artifact of a slow baseline. On makespan, R2 and RD are indistinguishable (14/14/22; mean paired difference +0.006, CI $[-0.007,\ 0.019]$; Wilcoxon $p = 0.12$), while RD rewrites more (0.618).

The reduction in rewriting is established only by R2 versus RA. On average the impact set excludes only $1 - 0.92$ of the unfinished occupancies (0.80–1.00 per cell; in 6 cells the two sets coincide). This small difference lowers the rewrite fraction from 0.604 to 0.540 (mean reduction 0.064, median 0.029, per cell 31/18/1 for R2 fewer/equal/more rewrites, $p = 8.8 \times 10^{-7}$; excluding the 6 identical cells, 31/12/1 with mean 0.073), while runtimes (2.74 vs. 2.76 ms) and makespans (mean ratio 0.999; R2 never worse) are nearly identical. These results must be read together. **Equal runtime shows that the millisecond response comes from single-pass rerouting, not from the impact set; equal makespan shows that the impact set is not an optimization device; the lower rewrite fraction is exactly the property guaranteed by Proposition 3.** At equal speed and makespan, forgoing the impact set rewrites about 6 percentage points more occupancies (about 12% relatively). The reduction grows as the impact set excludes more: 0.010 on `congested_8x4x4` (rewritable share 0.94) versus 0.118 on `S8x4x4_mid` (0.91). Figure 6 condenses this into a post hoc rule: the 23 cells with $\mathrm{Cl}/|R^{\circ}| \geq 0.93$ have a mean reduction of 0.029, and the 27 cells below it 0.094; above 0.97 the reduction is zero. The threshold is in-sample and not meant for extrapolation.

6.8 Sweep over disturbance magnitude

Table 10 sweeps φ on `congested_8x4x4` with escalation enabled (R0+ budget 2 s). Bounded repair is feasible in every cell, and even at $\varphi = 1$ its runtime stays on the order of ten milliseconds (2.2–23.8 ms over the sweep), without noticeable growth; R0+ attains the better $C_{\max}$ throughout. Runtime is not monotone in φ, possibly because small ratios need escalation more often while large ratios reroute more legs per round. On `example_3x3x2`, every cell is likewise feasible;

Table 10. Magnitude sweep (congested_8x4x4, means over 10 seeds; escalation enabled). "Time ratio": level mean of per-cell ratios of the R0+ budget to the STRC runtime; at the two smallest levels its within-level IQR exceeds the between-level variation.

φ	STRC feasible	STRC ms	R0+ $C_{\max}$	Time ratio
0.10	100%	9.1	103.5	406×
0.25	100%	11.1	107.0	325×
0.50	100%	8.6	111.8	333×
0.75	100%	8.4	125.8	254×
1.00	100%	10.7	148.1	203×

Table 11. E8: instance-size sweep (7 levels × 10 seeds). "Impact set": Cl/|R|, whose attainable upper bound is $|R^\circ|/|R| \approx 0.61$. Runtimes are medians; "RD/R2" is the ratio of the two median columns; the last column is the $C_{\max}$ win/tie/loss of R2 against RS.

Size	$\lvert R\rvert$	Impact set	R2 ms	RD ms	RD/R2	R2 vs. RS
8 × 4 × 4	148	0.561	2.9	4.4	1.5×	7/1/2
10 × 5 × 5	172	0.485	3.2	6.1	1.9×	9/0/1
12 × 6 × 6	230	0.481	4.5	10.7	2.4×	7/1/2
14 × 7 × 7	281	0.492	6.1	14.5	2.4×	8/0/2
16 × 8 × 8	291	0.534	6.4	16.4	2.6×	10/0/0
18 × 9 × 9	356	0.535	8.2	26.4	3.2×	10/0/0
20 × 10 × 10	402	0.476	10.4	34.2	3.3×	10/0/0

the per-cell ratio of the R0+ budget to the bounded-repair runtime ranges from 87× to 3051× over both instances, which is not a speedup at equal quality.

6.9 E8: extrapolation to larger instances

The key question is whether the impact-set share approaches 1 as the instance grows, which would make "bounded" meaningless. We use $2k$ jobs, k machines, and k vehicles for $k = 4, \ldots, 10$ (7 levels × 10 seeds = 70 pairs), with congestion, heterogeneity, flexibility, time ratio, and station cuts calibrated to fixed targets, so that only size changes (Table 11).

The share does not rise with size. Cl/|R| fluctuates between 0.48 and 0.56 without trend; relative to the unfinished occupancies it lies in 0.78–0.91, the same order as the 92% of the main batch. The impact set is thus not small, and seven levels do not support an asymptotic claim. **Runtime stays on the order of ten milliseconds.** As |R| grows 2.7-fold, the median runtime of R2 grows from 2.9 to 10.4 ms. Level-1 rerouting is feasible in 69/70 cells; the remaining cell succeeds in escalation round 3 (12.0 ms), giving 70/70. **The cost of global recomputation grows with size**: the RD/R2 ratio rises from 1.5× to 3.3×, not monotonically but by far more than any within-level IQR (at most 0.74). Against RS, R2 wins 7–10 of 10 cells per level. The sequence covers one congestion level and one instance seed.

6.10 Reproduction on the external-layout batch

On the 50 pairs of the second batch, E1, E2, E3, and E5 point in the same direction item by item: the scope on G_T is empty in 50/50, closure holds in 50/50, fields outside the set are unchanged in 50/50, and level-1 rerouting is feasible in 0/50 cells when scoping on G_T and in 50/50 when scoping on the impact set. The per-layout median share lies in 0.438–0.547 (main batch: pooled median 0.571). R0+ attains the better $C_{\max}$ at 17/18 budget points and ties at the remaining one, with no crossover. Hence the object-level results do not depend on our layout design; the only conclusion this batch changes is the one on structural predictability (Section 6.4).

7 Conclusion

When a flexible job shop records corridor occupancies with time windows, the operation-precedence graph contains no yielding relations between occupancies and cannot capture disturbances that invalidate only occupancies: under corridor blockage, operation-level scoping is empty although the original schedule is infeasible (50/50, C1). Delineating the scope on the reservation dependency graph makes the occupancies to be rewritten uniquely determined by the given relations rather than by a heuristic radius (50/50, 50/50, C2), and with the same rerouting and only the scope object changed, feasibility goes from 0/50 to 50/50 (C3). Rewriting all unfinished occupancies instead leaves runtime and makespan almost unchanged but raises the rewrite fraction from 0.540 to 0.604. Hence the millisecond response comes from single-pass rerouting, and **the benefit attributable to the impact set is that occupancies outside it are kept as planned** (Proposition 3); this benefit shrinks as the impact set approaches the set of all unfinished occupancies. Yielding relations, rerouting inside the set, and scope escalation are taken from existing practice.

7.1 Limitations and future work

We list four limitations in order of their impact, each followed by the corresponding direction.

(1) **Makespan.** Bounded repair is worse than warm-started global rescheduling at every tested budget, with no crossover (Section 6.5). Since neither method rewrites finished occupancies, the gap concerns $C_{\max}$ alone; narrowing it would require limited reassignment or resequencing, which drifts toward global rescheduling.

(2) **Strength and tightness of the guarantee.** Proposition 3 is only a sufficient condition, the edge set has not been verified exhaustively, and the impact set covers about 92% of the unfinished occupancies. The probe finds 0 leaks for the known abutting omission; other disturbance forms (shortened occupancies, longer delays) should be probed next. Tightness calls for a minimal rewritable set defined for a fixed repair capability and a quantified gap to the impact set. Static cut indicators cannot predict the impact-set size (Section 6.4); the next candidates are schedule-dependent, e.g., the number of overlapping occupancies on the disturbed corridor near t_{now} and its out-degree in the yielding graph.

(3) **Scope of the implementation.** Class B is handled by rerouting only, Class A adds vehicle swapping and reassignment without resequencing, and only a single disturbance is handled (A3). Size has been extended to 20 jobs and 10 machines, but not enough to support asymptotic claims. The next step is to relax A3 and examine whether the impact set inflates under continual disturbances and whether a restoration step is needed between repairs.

(4) **External reference points.** The public instance families [3, 6] publish machine-to-machine travel-time matrices; none of their 11 layouts can be reconstructed as a corridor graph consistent with the matrix, so R, G_R, $\mathrm{Cl}(\cdot)$, and Class B disturbances cannot be defined on them. Our second batch externalizes only the layouts [10]; travel times and job data are still our own, so the reference values of Lyu et al. or van Os [20] cannot be reproduced. An instance family publishing both exclusive corridors and per-leg durations is still needed, and a second implementation can be carried out together with such a reproduction.

Acknowledgments

This work was supported by <TO BE FILLED: funding program name and grant number>. The authors thank the anonymous reviewers for their comments on the experimental measures and the boundaries of the conclusions.

Conflict of interest statement

The authors declare no conflicts of interest related to this work.

References

[1] Riyad J. Abumaizar and Joseph A. Svestka. 1997. Rescheduling job shops under random disruptions. *International Journal of Production Research* 35, 7 (1997), 2065–2082. doi:10.1080/002075497195074

[2] M. Selim Aktürk and Elif Görgülü. 1999. Match-up scheduling under a machine breakdown. *European Journal of Operational Research* 112, 1 (1999), 81–97. doi:10.1016/S0377-2217(97)00396-2

[3] Ümit Bilge and Gündüz Ulusoy. 1995. A Time Window Approach to Simultaneous Scheduling of Machines and Material Handling System in an FMS. *Operations Research* 43, 6 (1995), 1058–1070. doi:10.1287/opre.43.6.1058

[4] Andrea D'Ariano, Dario Pacciarelli, and Marco Pranzo. 2007. A branch and bound algorithm for scheduling trains in a railway network. *European Journal of Operational Research* 183, 2 (2007), 643–657. doi:10.1016/j.ejor.2006.10.034

[5] Jie Fu, Bin Yang, Zhixing Chang, Yuanrong Zhang, Jiarui Wang, Xiaotong Wang, and Lei Wang. 2025. Integrated Production–Logistics Scheduling in Flexible Assembly Shops Using an Improved Genetic Algorithm. *Machines* 13, 12 (2025), 1090.

[6] Seyed Mahdi Homayouni and Dalila B. M. M. Fontes. 2021. Production and Transport Scheduling in Flexible Job Shop Manufacturing Systems. *Journal of Global Optimization* 79, 2 (2021), 463–502. doi:10.1007/s10898-021-00992-6

[7] Jeonghyeon Kim and Junwoo Kim. 2026. Congestion-Aware Scheduling for Large Fleets of AGVs Using Discrete Event Simulation. *Electronics* 15, 1 (2026), 139. doi:10.3390/electronics15010139

[8] Jürgen Kuster, Dietmar Jannach, and Gerhard Friedrich. 2007. Local Rescheduling: A novel approach for efficient response to schedule disruptions. In *2007 IEEE Symposium on Computational Intelligence in Scheduling (CISched)*. IEEE, Honolulu, HI, USA, 79–86. doi:10.1109/SCIS.2007.367673

[9] Ruiqi Li, Jianlin Mao, Xing Wu, Wenna Zhou, Chengze Qian, and Haoshuang Du. 2026. A New Framework for Job Shop Integrated Scheduling and Vehicle Path Planning Problem. *Sensors* 26, 2 (2026), 543. doi:10.3390/s26020543

[10] Xiangfei Lyu, Yuchuan Song, Changzheng He, Qi Lei, and Weifei Guo. 2019. Approach to Integrated Scheduling Problems Considering Optimal Number of Automated Guided Vehicles and Conflict-Free Routing in Flexible Manufacturing Systems. *IEEE Access* 7 (2019), 74909–74924. doi:10.1109/ACCESS.2019.2919109

[11] Nir Malka, Guy Shani, and Roni Stern. 2024. Online Planning for Multi Agent Path Finding in Inaccurate Maps. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, Abu Dhabi, United Arab Emirates, 10214–10221. doi:10.1109/IROS58592.2024.10801975

[12] Alessandro Mascis and Dario Pacciarelli. 2002. Job-shop scheduling with blocking and no-wait constraints. *European Journal of Operational Research* 143, 3 (2002), 498–517. doi:10.1016/S0377-2217(01)00338-1

[13] Leilei Meng, Weiyao Cheng, Chaoyong Zhang, Kaizhou Gao, Biao Zhang, and Yaping Ren. 2025. Novel CP Models and CP-Assisted Meta-Heuristic Algorithm for Flexible Job Shop Scheduling Benchmark Problem with Multi-AGV. *IEEE Transactions on Systems, Man, and Cybernetics: Systems* 55, 11 (2025), 8455–8468. doi:10.1109/TSMC.2025.3604355

[14] Djamila Ouelhadj and Sanja Petrovic. 2009. A Survey of Dynamic Scheduling in Manufacturing Systems. *Journal of Scheduling* 12, 4 (2009), 417–431. doi:10.1007/s10951-008-0090-8

[15] Haoxiang Qin, Yi Xiang, Yuyan Han, Yuting Wang, Junqing Li, and Quanke Pan. 2026. A Knowledge Region Selection Enhanced Quality-Diversity Algorithm for Real-World Flexible Job Shop Scheduling with Automated Guided Vehicles Transportation. *Engineering Applications of Artificial Intelligence* 164 (2026), 113352. doi:10.1016/j.engappai.2025.113352

[16] Velusamy Subramaniam and Amritpal Singh Raheja. 2003. mAOR: A heuristic-based reactive repair mechanism for job shop schedules. *The International Journal of Advanced Manufacturing Technology* 22, 9-10 (2003), 669–680. doi:10.1007/s00170-003-1601-6

[17] Jiachen Sun, Zifeng Xu, Zhenhao Yan, Lilan Liu, and Yixiang Zhang. 2023. An Approach to Integrated Scheduling of Flexible Job-Shop Considering Conflict-Free Routing Problems. *Sensors* 23, 9 (2023), 4526. doi:10.3390/s23094526

[18] Jiří Švancara, Marek Vlk, Roni Stern, Dor Atzmon, and Roman Barták. 2019. Online Multi-Agent Pathfinding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 33. AAAI Press, Honolulu, HI, USA, 7732–7739. doi:10.1609/aaai.v33i01.33017732

[19] Hongtao Tang, Xuewei Zhang, Jiawei Huang, Xuesong Xu, and Jiansha Lu. 2026. Integrated Scheduling of Multi-Objective Lot-Streaming Hybrid Flowshop with AGV under Dynamic Environments. *International Journal of Industrial Engineering Computations* 17, 2 (2026), 295–316. doi:10.5267/j.ijiec.2025.9.003

[20] Roel W. M. van Os and Twan Basten. 2026. An Exact Decomposition-Based Approach to the Conflict-Free-Transportation-Constrained Flexible Job-Shop Scheduling Problem. *Computers & Operations Research* 187 (2026), 107342. doi:10.1016/j.cor.2025.107342

[21] Guilherme E Vieira, Jeffrey W Herrmann, and Edward Lin. 2003. Rescheduling Manufacturing Systems: A Framework of Strategies, Policies, and Methods. *Journal of Scheduling* 6, 1 (2003), 39–62. doi:10.1023/A:1022235519958

[22] Bin Xin, Sai Lu, Qing Wang, and Fang Deng. 2025. A Review of Flexible Job Shop Scheduling Problems Considering Transportation Vehicles. *Frontiers of Information Technology & Electronic Engineering* 26, 3 (2025), 332–353. doi:10.1631/FITEE.2300795

[23] Yannik Zeiträg and José Rui Figueira. 2025. A novel machine-learning rolling horizon heuristic for dynamic lot-sizing and job shop scheduling problems. *International Journal of Production Research* 63, 12 (2025), 4563–4589.

[24] Hongliang Zhang, Chaoqun Qin, Wenhui Zhang, Zhenxing Xu, Gongjie Xu, and Zhenhua Gao. 2023. Energy-Saving Scheduling for Flexible Job Shop Problem with AGV Transportation Considering Emergencies. *Systems* 11, 2 (2023), 103. doi:10.3390/systems11020103

[25] Yulun Zhang, He Jiang, Varun Bhatt, Stefanos Nikolaidis, and Jiaoyang Li. 2024. Guidance Graph Optimization for Lifelong Multi-Agent Path Finding. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI)*, Kate Larson (Ed.). International Joint Conferences on Artificial Intelligence Organization, Jeju, South Korea, 311–320. doi:10.24963/ijcai.2024/35

[26] Ziyan Zhao, Shixin Liu, MengChu Zhou, Xiwang Guo, and Liang Qi. 2019. Decomposition method for new single-machine scheduling problems from steel production systems. *IEEE Transactions on Automation Science and Engineering* 17, 3 (2019), 1376–1387.